TECHNICAL RESEARCH REPORT

NotPetya / ExPetr

CAPE Task 4 | Execution Evidence and Readable Control-Flow Reconstruction

Case scope: one Windows x86 DLL execution captured by CAPE on 6 November 2026 (host timestamp); reviewed against the provided Ghidra decompilation and static triage. The goal is to explain why the VM restarted normally, not to reproduce destructive behavior.

CASE	VALUE
Submitted sample	**NotPetya.dll** (PE32, Intel 80386, DLL)
SHA-256	027cc450ef5f8c5f653329641ec1fed91f694e0d229928963b30f6b0d7d3a745
Task / environment	CAPE task 4; cuckoo1; package dll; INetSim route
Capture length	452 seconds; start 2026-11-06 22:24:51; end 22:32:23
CAPE automated verdict	Malicious, score 10.0 / 10
Available primary records	report.json (77,732,448 bytes), analysis.log, cuckoo.log; source decompilation and triage

Core finding

NotPetya reached its exported entry point and performed malicious activity. The submitted telemetry **does not demonstrate a successful raw-disk write or bootloader installation**. Two denied token-opening requests are a plausible contributing factor, not conclusive proof of the boot-stage failure.

Research outputs: This report and **NotPetya_Reconstructed_Annotated_Flow_EN.c**, an English, intentionally non-compilable analytical map with original Ghidra function references.

Source integrity note: the uploaded gzip archive is truncated. All quantitative CAPE findings in this report come from recovered structured records; unextracted EVTX, dumps, PCAP bytes, and screenshots have not been independently reviewed.

1. Executive determination

The sample is not a dormant or incorrectly invoked DLL. CAPE recorded the primary **rundll32.exe** process (PID **4900**) and the expected ordinal-1 loading path. The process opened its own DLL, overwrote it, dropped a PE-like auxiliary file, created multiple threads, performed cryptographic API calls, attempted network connections, and initiated the Windows Task Scheduler command chain.

QUESTION	ASSESSMENT	SUPPORT
Did the core DLL stage execute?	**Yes**	rundll32.exe #1; process events attributed to PID 4900.
Was a reboot requested/scheduled?	**Attempted**	cmd.exe -> schtasks.exe, command creating a one-time shutdown task.
Did file-crypto APIs execute?	**Yes**	346 successful CryptEncrypt calls in PID 4900.
Did external movement succeed?	**Not demonstrated**	191 connect calls returned failure; no reconstructed TCP sessions.
Was the MBR/boot stage installed?	**Not demonstrated**	No observed PID 4900 disk-device write matching PhysicalDrive0.
Why did the VM reboot normally?	**Open investigation**	Privilege gate and device-write path are candidates; not proven.

Confidence language

Observed: an action appears in attributable CAPE API/process/file telemetry. **Static:** the code or string exists in the decompiled sample but may not have run. **Inferred:** a causal explanation consistent with evidence but missing a decisive observation. **Not observed:** absent from available telemetry; this is not an absolute claim of impossibility.

Interpretive correction to the preliminary chat diagnosis

The two **STATUS_ACCESS_DENIED** token requests cannot establish that all privilege acquisition failed: PID 4900 also had **two successful NtOpenProcessToken calls**. The original code tests named privileges via AdjustTokenPrivileges and GetLastError. A failed NtOpenProcessToken call is relevant but does not reveal the final global privilege mask.

Bottom line

The clean reboot is consistent with an unsuccessful boot-stage modification, but direct proof requires boot-sector comparison, complete disk I/O telemetry, or a verified runtime privilege-mask value. The observed activity already proves the main DLL executed.

2. Materials, methodology and completeness

Files examined

SOURCE	WHAT IT SUPPORTS
cape-task4.tar.gz	Recovered CAPE report.json, analysis.log and cuckoo.log. The gzip stream terminates unexpectedly.
NotPetya.dll.c	12,031-line Ghidra-generated C-like decompilation of the PE32 DLL, including original FUN_... addresses.
notpetya(1).txt	Static triage: PE header, entropy, embedded PE/zlib signatures, imports, strings, hash.

The CAPE structured report includes 33 monitored process entries, 25 process-memory metadata records, 56 signature detections, 102 dropped-file records, and 199 CAPE payload/extraction entries. **These totals are not counts of malware-created components.** Some records are clearly CAPE analyst/analyzer scripts, Windows updates and ordinary background activity. Attribution below focuses on PID 4900 and its immediate child-process chain.

Static metadata

PARAMETER	VALUE
File architecture	PE32 DLL / Intel i386 / 5 sections
File size	362,360 bytes
File entropy	7.801934 bits per byte (high; compressed/packed contents plausible)
Embedded signatures	Additional PE signatures at offsets 0x14820 and 0x16060; zlib candidates in the static triage
PE signatures	PKCS#7 / X.509 data detected. CAPE also flags invalid Authenticode signature; embedded certificate data is not proof of valid signing.

Acquisition limitation

The archive is approximately 77 MB compressed and cannot be fully validated with gzip/tar. Its surviving files contain enough event data to reconstruct substantial activity, but not enough to inspect every claimed dumped image, memory region, screenshot, or original network capture. A report.JSON reference to an artifact does not mean its bytes were independently verified in this review.

All artifacts were processed as **data**. No suspicious executable, DLL, resource, dropped PE, or payload was run in preparing these deliverables.

3. Dynamic event chronology

The guest monitor uses a clock that is several hours ahead of the host task timestamps. The sequence below reproduces the guest-recorded times without treating them as the host timezone.

GUEST TIME	PID	EVENT / RESULT
01:25:09.723	4900	rundll32.exe first observed.
01:25:10.004	4900	NtOpenProcessToken access 0x08 succeeds; access 0x80 denied.
01:25:10.192	4900	NtOpenProcessToken access 0x28 denied; DLL file opened.
01:25:10.270	4900	Original Temp\NotPetya.dll overwritten; zero-prefixed buffer (362,360-byte write).
01:25:10.363	4900	NtOpenProcessToken access 0x0A succeeds.
01:25:10.551	4900	C:\ProgramData\dllhost.dat created; 381,816 bytes written; buffer starts MZ.
01:25:10.598	4900	Write to named pipe srvsvc (72-byte buffer).
01:25:10.707-0.817	4900	Named-pipe wkssvc interactions; two 116-byte writes.
01:25:10.795	8952	cmd.exe child observed.
01:25:11.146	6572	schtasks.exe child of cmd.exe observed.
01:25:53.067	4900	Attempted administrative-share file write to 192.168.100.1 fails with 0xC00000BE.

The direct child chain and scheduled-task command appear in the behavior process tree/summary. Do not confuse the command being launched with confirmed successful task registration or subsequent malicious boot code execution.

Observed process chain

```
rundll32.exe     PID 4900   [NotPetya.dll export ordinal #1]
   +-- cmd.exe   PID 8952   [task-creation command]
          +-- schtasks.exe PID 6572

Other 30 reported processes are not automatically malware children:
many are svchost.exe / taskhostw.exe / updater.exe / EdgeUpdate / OneDrive.
```

4. Direct file and API evidence

Writes attributable to the NotPetya PID

TARGET	BYTES	INTERPRETATION
Temp\NotPetya.dll	362,360	Self-image overwrite with zero-leading data; deletion attempted in code.
C:\ProgramData\dllhost.dat	381,816	PE-like auxiliary binary; starts with MZ. Exact role requires extracted-binary inspection.
\Device\NamedPipe\srvsvc	72	RPC / local named-pipe traffic; not evidence by itself of remote compromise.
\Device\NamedPipe\wkssvc	116 x 2	Two named-pipe writes, consistent with network-service interaction.

API-call counts for PID 4900

API	COUNT	SUCCESS / RETURN
CryptEncrypt	346	346 success (BOOL return = 1)
connect	191	0 successes (all return SOCKET_ERROR)
NtCreateFile	367	362 success; 5 failures
NtWriteFile	5	5 success (2 files, 3 pipes)
NtOpenProcessToken	4	2 success; 2 access denied
CreateThread	8	8 recorded success
DeviceIoControl	0	No monitored calls under PID 4900

Successful CryptEncrypt invocations demonstrate use of a cryptographic API but do not prove that 346 distinct victim files were encrypted. Similarly, a successful NtWriteFile proves a monitored write but not the integrity of all resulting file bytes; validation requires the corresponding file artifacts.

Automated detection versus independently demonstrated action

CAPE produced 56 behavioral/signature findings, including mass encryption, scheduled task, overlay, API resolution, token manipulation, and unbacked-memory behaviors. These findings should be triaged by originating PID, source event, and callsite before attributing all of them to NotPetya; the environment also recorded substantial normal Windows and CAPE instrumentation activity.

5. Privilege logic and the missing boot stage

What the Ghidra reconstruction establishes

At **FUN_10007cc0**, the sample calls **FUN_100081ba** for SeShutdownPrivilege, SeDebugPrivilege and SeTcbPrivilege, accumulating successful results into **DAT_1001f144**. **FUN_10007deb** gates disk-related functions on bit 1 (SeDebugPrivilege in the reconstructed mask). The relevant source-level control flow is:

```
FUN_10007cc0():
  mask = try_enable(SeShutdownPrivilege) ? 0x01 : 0;
  if (try_enable(SeDebugPrivilege)) mask |= 0x02;
  if (try_enable(SeTcbPrivilege))   mask |= 0x04;
  DAT_1001f144 = mask;

FUN_10007deb():
  if (DAT_1001f144 & 0x02) {
      FUN_1000835e();   // system marker branch
      FUN_10008d5a();   // raw volume/disk branch
  }
  FUN_100084df();       // scheduling branch continues
```

The code excerpt is a **source interpretation**, not a value trace. Although the CAPE report shows denied token opens (0x80, 0x28), it also shows successful opens (0x08, 0x0A). No run-time read of DAT_1001f144 was preserved in the accessible material.

Raw disk path exists statically

FUN_10008d5a references **\\.\C:**; **FUN_10008cbf** references **\\.\PhysicalDrive0**. They contain raw-device opening, control and write operations in the Ghidra output. The decompiler shows suspicious handle checks and locally uninitialized variables; these are reasons not to assume the reconstructed text is bit-perfect, much less compile-ready source.

What is missing dynamically

For PID 4900, the structured API trace contains **no DeviceIoControl call** and no NtWriteFile target identified as PhysicalDrive0 or the raw C: volume. The report contains later physical-device reads from unrelated svchost.exe activity, which is not evidence of NotPetya disk modification. An API monitor can miss events, so absence here is a limitation of available evidence, not a mathematical proof of no write.

Most defensible conclusion

The code contains a privilege-gated raw-disk branch. The captured execution reached major other branches, but the available API events do not show a successful destructive disk operation. The exact failed condition remains unresolved without a runtime flag value and independent disk state evidence.

6. Cryptography, extraction and boot semantics

File-path-oriented cryptography

FUN_10001eef enumerates logical drives, examines their types, and creates workers for fixed drives. The Ghidra source includes a public RSA key string and Windows CryptoAPI usage. CAPE attributes 346 successful CryptEncrypt calls to the primary process. Without memory/output artifact correlation, the captured API sequence does not establish which specific file names were irreversibly affected.

Boot-stage code versus evidence of boot-stage activation

The compiled sample includes boot-screen text and code references to disk/volume operations. These are **capabilities present in the binary**. They do not imply that the guest VM loaded the malicious boot stage during this Task 4 execution. The observed normal Windows restart is compatible with the available telemetry, but the missing transition still requires verification.

Auxiliary PE-like resource

At 01:25:10.551, the sample wrote 381,816 bytes to C:\ProgramData\dllhost.dat. Its buffer began with **MZ**, consistent with a PE header. **FUN_10008999** retrieves a resource and writes a derived output file. This static-to-dynamic correspondence supports a resource-extraction interpretation, without proving what the helper later executed.

Certificate and packer observations

The static triage found two embedded PE signatures, several zlib data signatures and a PKCS#7 / certificate block. CAPE reported an invalid Authenticode signature. The presence of a Microsoft certificate chain in parsed data must not be described as a legitimate Microsoft signature on NotPetya.

Practical consequence for this case

The better research question is not whether the sample ever ran - it did - but exactly which earlier branch or environment state prevented the raw-device stage from being confirmed. The attached C-shaped reconstruction marks destructive and propagation operations as analysis-only so that the control flow remains readable without recreating an operational wiper.

7. Network behavior and attribution

The original process made **191 connect()** calls, all failing according to CAPE. The network parser reconstructed **0 TCP sessions**, **0 HTTP requests** and **162 UDP records** in the task-wide view. It also reported three DNS queries. A task-wide packet capture cannot be automatically assigned wholesale to PID 4900.

ITEM	OBSERVATION
Primary failure	NtCreateFile on \??\UNC\192.168.100.1\admin$\NotPetya.dll failed (0xC00000BE).
Named-pipe evidence	svc named pipes srvsvc and wkssvc written by PID 4900; consistent with RPC/network discovery.
Task-wide dead hosts	192.168.100.1 on ports 445, 80 and 53.
DNS examples	www.msftconnecttest.com; searchapp.bundleassets.example; -keyid-...microsoftaik.azure.net.
Attribution	No observed successful remote share write or confirmed lateral compromise in this run.

Network discoveries should be represented as **attempted** connection or attempted lateral activity, not proven worm spread. In particular, the task-wide DNS list is compatible with background Windows connectivity checks, so it cannot be assigned to NotPetya without process-level attribution.

How to interpret the 199 CAPE extraction entries

Numerous CAPE “payload” entries are Python source/bytecode from the sandbox guest analyzer, and much of the 102-file dropped list consists of instrumentation, logs and background OS artifacts. These should be separated from the PE-like dllhost.dat, original DLL and verifiable malware-owned artifacts. Raw totals without provenance are misleading.

8. Case chronology and event provenance

All timestamps in the forensic tables below are copied from CAPE process-event records. The host scheduler records the analysis on **6 November 2026**, whereas monitored guest API records are stamped **7 November 2026 at 01:25**. The apparent difference reflects the environment clocks/time-zone presentation. Without an independent synchronized clock measurement, subsecond event ordering within the same monitored process is more defensible than comparisons across host and guest services.

Reproducible scope and definitions

FIELD	EXACT SCOPE	INTERPRETIVE RULE
Primary PID	4900, rundll32.exe, 11,889 CAPE calls	Use when making statements about malware-associated API activity.
Immediate children	8952 (cmd.exe), 6572 (schtasks.exe)	Use parent ID and command-line continuity, not generic tool name.
Other processes	30 other monitored entries	Treat as background unless independent lineage proves otherwise.
Host task duration	452 seconds, with timeout=True	Not interchangeable with the guest analyzer requested 300 seconds.
CAPE signatures	56 signature detections	A signature is a classifier result, not standalone proof of all its claims.
Memory artifacts	25 procmemory records	Metadata visible; the damaged archive does not provide every payload for independent bytes-level verification.

Precisely timestamped evidence points

GUEST TIME	EVENT	OBSERVED FACT
01:25:09.723	Primary process first observed	PID 4900, monitored 32-bit rundll32.exe instance.
01:25:10.004	Access-token work	First token open succeeds; second request denied.
01:25:10.192	Permission-related API failure	NtOpenProcessToken with 0x28 requested and denied.
01:25:10.270	Original image modified	NtWriteFile success against staged NotPetya.dll; zero-filled buffer.
01:25:10.317	Child process created	cmd.exe launched to invoke Task Scheduler.
01:25:10.535	Child thread resumed	NtResumeThread success for the just-created cmd.exe.
01:25:10.551	Auxiliary file written	NtWriteFile success to C:\ProgramData\dllhost.dat, 381,816 bytes.
01:25:10.598	Crypto provider/key activity	CryptAcquireContextW and CryptGenKey success.
01:25:10.598	First named-pipe write	72-byte srvsvc record; observe RPC-style exchange, not an infection.
01:25:10.738	Additional pipe event	116-byte write to wkssvc pipe.
01:25:10.817	Additional pipe event	Second observed 116-byte write to wkssvc pipe.
01:25:53.067	Administrative share failure	UNC admin$ file open returned 0xC00000BE.

Important: the scheduler command was created as a process invocation. Whether Windows successfully persisted or subsequently executed the scheduled task is a separate proposition. Likewise, evidence of named-pipe writes cannot be equated with successful propagation.

9. Invocation, loaded image, and primary dispatcher

DLL startup versus ordinal export

The record distinguishes a DLL_PROCESS_ATTACH callback from the exported execution dispatcher. Ghidra labels the DLL entry function `entry` near 0x10007d39 and the export `FUN_10007deb`. CAPE environment strings show the primary process command line invoking **ordinal #1** through a 32-bit rundll32 process. This is both a source-level and execution-level link; it is not inferred merely from the PE being a DLL.

```
rundll32.exe  ->  NotPetya.dll, #1
  [Ghidra entry]  stores module state; suppresses DLL thread callbacks
  [0x10007deb]    calls 0x10007cc0 (initialization)
                    computes privilege mask
                    evaluates conditional disk path
                    schedules reboot attempt
                    starts discovery and auxiliary workers
                    processes delay gates and fixed-volume scan
```

Dispatcher branch inventory

CONTROL POINT	SOURCE-LEVEL CONDITION	RUNTIME EVIDENCE
0x10007cc0	Initialize privilege mask and runtime/context flags	Token events; code path invoked by main dispatcher.
0x10009590	Conditional relocation of DLL image in memory	Self-image access; memory protection changes.
0x10008d5a	Disk-stage helper guarded by debug-privilege bit	No attributable PhysicalDrive0 write in PID 4900.
0x100084df	Schedule reboot through operating-system utility	cmd.exe -> schtasks.exe observed.
0x10007c10	Start host discovery/network background thread	Network API attempts; worker thread addresses.
0x10007545	Auxiliary resource execution path subject to flags	Resource writing; exact substage requires recovered binary.
0x10008999	Resource extraction / file drop depending on flags	dllhost.dat write observed.
0x10001eef	Enumerate fixed drives under time/environment gate	CryptEncrypt calls, file discovery; no per-file outcome proven.

Why the entry point alone cannot explain outcomes

The decompilation has multiple early and delayed gates, external resource dependencies, asynchronous threads and process-level side effects. A success signal from a single early branch is not sufficient to assert that the later boot modification completed. The relocated code and in-memory image handling deserve independent examination, particularly because Ghidra decompilation can misassign local stack variables and make a data-dependent call look unconditional.

10. Primary process API workload

The table gives exact CAPE call counts for the main PID only. Relative frequency helps distinguish high-volume directory traversal and section mapping from one-off security and process-control events. Counts are not normalized for CAPE instrumentation, API aliases or repeated system-library activity.

API	CALLS	INTERPRETATION
`NtClose`	4196	Handle cleanup across phases
`FindFirstFileExW`	3201	Directory traversal; exact file selection unknown
`NtMapViewOfSection`	371	Memory mapped views
`NtCreateFile`	367	Open filesystem/pipe objects
`NtCreateSection`	366	Create memory/file sections

API	CALLS	INTERPRETATION
NtUnmapViewOfSectionEx	348	Remove memory mapped views
NtQueryInformationFile	348	File metadata reads
CryptEncrypt	346	Cryptographic API usage, not file count
LdrGetProcedureAddressForCaller	246	Dynamic procedure resolution
socket	192	Socket creation
ioctlsocket	192	Socket option/behavior
connect	191	Connection attempts
closesocket	190	Socket cleanup
select	189	Socket readiness checks
NtDeviceIoControlFile	187	Device requests, primarily NSI in observed calls
NtProtectVirtualMemory	129	Memory protection transitions
Process32NextW	117	Process enumeration
NtQueryInformationToken	13	Security-token queries
NtCreateThreadEx	9	Thread creation
NtWriteFile	5	Five writes scoped to specific handles
NtOpenProcessToken	4	Four attempts, two denied
NtCreateUserProcess	1	Command-child creation
NtWriteVirtualMemory	1	Small remote process memory write
NtResumeThread	1	Resume child thread
CryptGenKey	1	Key generation
CryptAcquireContextW	1	Crypto provider initialization

Interpretation of frequent file activity

The dominant `FindFirstFileExW` call count is consistent with file-discovery activity. It is **not** a count of distinct encrypted files, nor does it establish successful full-volume encryption. The 346 successful CryptEncrypt calls likewise prove use of CryptoAPI but cannot be equated with 346 encrypted victims without correlating the affected buffer, file handle, target and later write for each operation.

11. Privilege initialization and denial evidence

What the code tests

The 0x10007cc0 routine tests SeShutdownPrivilege, SeDebugPrivilege, and SeTcbPrivilege via a helper at 0x100081ba. It stores success bits into DAT_1001f144. The raw-disk helper branch at 0x10007deb is guarded by the bit corresponding to the debug-privilege test. This is an **actual conditional** in the submitted Ghidra output.

Four primary NtOpenProcessToken records

TIME	DESIRED ACCESS	RESULT	HANDLE
01:25:10,004	0x00000008	SUCCESS	0x0000014c
01:25:10,004	0x00000080	0xffffffffc0000022	0x00000000
01:25:10,192	0x00000028	0xffffffffc0000022	0x00000000
01:25:10,363	0x0000000a	SUCCESS	0x000003b0

The host analyzer log explicitly states `analysis running as an admin`. The statement is compatible with the denied requests: running with administrator elevation does not guarantee that every requested token access right or privilege enablement succeeds in the way the sample expects. It would be a logical error to infer from these four calls alone that the global privilege mask equals zero.

Three different propositions

QUESTION	AVAILABLE EVIDENCE	STATUS
Was a token handle ever opened?	Two NtOpenProcessToken calls succeeded.	Yes
Did any token opens fail?	Two calls returned 0xC0000022.	Yes
Did SeDebugPrivilege ultimately become enabled?	No captured DAT_1001f144 runtime value or definitive AdjustTokenPrivileges return context.	Unknown
Did the disk writer run?	No PID 4900 raw-disk write recorded.	Not established

Testing the causal hypothesis without wishful inference

A causally sufficient explanation needs at least one additional piece of evidence: the runtime privilege mask at the dispatcher branch, a trace of the branch taken, or recorded disk-device open/write outcomes from the disk helper itself. Even with SeDebugPrivilege enabled, failures in device open or I/O operations, platform boot layout or instrumentation gaps could account for the later normal reboot. The present conclusion is an **unresolved boot-stage outcome**, not a proven privilege failure.

12. Filesystem activity and self-erasure

The five actual NtWriteFile records in PID 4900

GUEST TIME	HANDLE NAME / TARGET	REPORTED LENGTH	STATUS
01:25:10,270	C:\Users\victor\AppData\Local\Temp\NotPetya.dll	362360	success
01:25:10,551	C:\ProgramData\dllhost.dat	381816	success
01:25:10,598	\Device\NamedPipe\srvsvc	72	success
01:25:10,738	\Device\NamedPipe\wkssvc	116	success
01:25:10,817	\Device\NamedPipe\wkssvc	116	success

The first write targets the staged copy of NotPetya.dll under the user temp directory and carries zero-filled data in the logged buffer. Its reported length matches the original 362,360-byte sample. The second write produces `C:\ProgramData\dllhost.dat` with a reported length of **381,816 bytes**; its logged buffer begins with an MZ signature. The remaining writes use named-pipe handles and are not disk-sector writes.

The critical distinction: handle identity

A low-level file-write API can target an ordinary file, a named pipe, a raw volume, or a physical drive. The CAPE event explicitly supplies HandleName. Consequently, counting every NtWriteFile as a destructive disk operation would be a categorical mistake. None of these five writes is attributed to `\\.\PhysicalDrive0`. This is more specific than saying the program merely imports WriteFile.

Other related file events

Primary NtCreateFile requests to the original staged DLL succeeded; the auxiliary file was opened for output; a later administrative-share open referencing `\??\UNC\192.168.100.1\admin$\NotPetya.dll` failed with `0xC00000BE`. The latter is evidence of an attempted remote administrative-share operation, not proof that the malware copied itself to a second computer.

Background and analyzer contamination

The report contains numerous Python source, bytecode, screenshot and module files under the CAPE analyzer directory. CAPE also reports normal Windows maintenance/updater binaries. These records must be separated from native NotPetya output by handle, process lineage, write direction, type and runtime timestamp. It is incorrect to describe all 102 dropped-file metadata entries as dropped malware payloads.

13. Embedded content and unpacking hypotheses

The static triage flags two additional PE signatures at file offsets 0x14820 and 0x16060, and several zlib-compressed-data candidates. It also identifies PKCS#7 / X.509 structures in the trailing portion. Such signatures establish byte patterns; without an extracted, independently hashed payload, they do not establish a complete mapping between these signatures and runtime components.

STATIC OBSERVATION	LOCATION	EVIDENTIARY VALUE
Main PE header	0x00000	Top-level DLL image

STATIC OBSERVATION	LOCATION	EVIDENTIARY VALUE
Secondary PE signature	0x14820	Embedded executable candidate
Secondary PE signature	0x16060	Additional embedded executable candidate
Compression candidate	0x19AEC	zlib stream candidate
Compression candidate	0x1FC6C	zlib stream candidate
Compression candidate	0x26790	zlib stream candidate
Compression candidate	0x55408	zlib stream candidate
PKCS#7 signature candidate	0x57008	Certificate container; not validation of trusted signature

The event that wrote dllhost.dat strongly suggests an extracted PE-format helper. But the damaged archive does not provide an independently reviewed copy of its raw bytes, so the report does not assign it definitive capabilities or a cryptographic hash. It should be recovered from a validated CAPE bundle and checked against the sample resources before stronger assertions.

Image relocation and self-cleanup mechanism

Ghidra `FUN_10009590` includes virtual memory allocation and protection changes. `FUN_100094a5` includes accesses to the original DLL image and overwrite/delete-related operations. Their apparent relationship with the first NtWriteFile and section-mapping telemetry is plausible, but not a proof of every particular in-memory relocation branch. Reconstructed C names are hypotheses linked to source addresses, not recovered original identifiers.

14. Crypto activity: what is proven, and what is not

The primary process performed one successful `CryptAcquireContextW`, one successful `CryptGenKey`, and 346 successful `CryptEncrypt` operations, per CAPE API results. The observed key-generation algorithm identifier is `0x660E`. This is consistent with the Windows CALG_AES_128 identifier; however, the report cannot infer all subsequent buffer-level encryption parameters or the intended recovery workflow from that identifier alone.

CryptoAPI evidence table

MEASUREMENT	COUNT / VALUE	INTERPRETATION
CryptAcquireContextW	1 success	Provider context acquired
CryptGenKey	1 success, Algid 0x660E	Key generation occurred
CryptEncrypt	346 successes	A sequence of encryption operations executed
Encrypted files	Not countable from this table	Requires handle/buffer correlation and after-image
Encrypted MFT	Not established	Separate boot-stage question
Data recovered from key	Not established	No demonstration of recoverability from source telemetry

Per-volume code vs observed buffer operations

The static dispatcher at `FUN_10001eef` enumerates logical drives and selects drive types. A worker at `FUN_10001e51` reaches a CryptoAPI sequence through subordinate helpers. These are relevant source links, but a sampled CryptEncrypt event does not carry enough independent information to bind one specific filesystem object to an encrypted output. The report therefore avoids fabricating filenames or claiming the 346 calls represent completed infections.

Why the analyzer directory matters

The CAPE behavioral summary includes the analyzer extraction directory among accessed targets. That makes it essential to distinguish intentional targeting, traversal artifacts and collection-side bookkeeping. File name strings appearing in an encryption-related trace are not automatically proof that those files were irreversibly transformed. A complete answer requires the recovered full data of writes, file content comparisons, and precise handle-to-file tracking.

15. Propagation and unsuccessful connections

The primary process made 191 `connect` calls and the monitor marked all of them unsuccessful. The parser reconstructed zero TCP sessions and zero HTTP requests from the reported PCAP view. Independently, the network summary lists 162 UDP datagrams and three DNS requests. These numbers measure distinct telemetry surfaces, not three different confirmations of successful worm spread.

NETWORK SURFACE	OBSERVED VALUE	PROPER CLAIM
Primary PID socket()	192 calls	Socket handles created
Primary PID connect()	191 calls, 191 failures	Network destinations attempted, not reached
Primary PID select()	189 calls	Readiness polling is present
Network parser TCP	0 sessions	No reconstructed TCP flows
Network parser HTTP	0 requests	No reconstructed HTTP transfers

NETWORK SURFACE	OBSERVED VALUE	PROPER CLAIM
Network parser UDP	162 datagrams	UDP traffic present, source attribution needed
DNS requests	3 entries	Names resolved/queried during VM run, not all malware-owned
Remote admin$ path	One failed open	Remote copy intention, no established copy

Named-pipe evidence

The sampled process wrote a 72-byte message to an srvsvc named pipe and two 116-byte messages to a wkssvc named pipe. These are consistent with RPC-style service-discovery or remote-service handling, but the available trace is insufficient to confirm successful authentication or lateral movement. Their source handles and bytes can be inspected from CAPE in a read-only forensic workflow; no network action is necessary to interpret the records.

No proof of C2 or an external downloader

The structured record does not establish a command-and-control session or HTTP delivery. The campaign may have multiple infection mechanics historically, but this single observed Task 4 does not prove downstream exploitation, credential theft, a remote host compromise or payload retrieval. The absence of a reconstructed flow is also partly a function of network isolation and capture coverage.

16. Threading, suspended process creation and memory manipulation

The main PID created several worker threads and one user process. The record contains an NtCreateUserProcess success for cmd.exe, followed by a successful small NtWriteVirtualMemory operation directed to that process and a successful NtResumeThread. This supports an analysis of process-creation staging. It is **not**, by itself, a confirmed malicious code injection into an unrelated external target.

CONTROL EVENT	COUNT	KEY EVIDENCE
`NtCreateUserProcess`	1	One child process; shell interpreter
`NtWriteVirtualMemory`	1	One remote memory write, 4 bytes
`NtResumeThread`	1	One successful thread resume
`NtCreateThreadEx`	9	Multiple worker threads
`CreateThread`	8	High-level worker creation
`NtProtectVirtualMemory`	129	Protection adjustments in monitored address spaces
`NtCreateSection`	366	Section-backed memory mapping
`NtMapViewOfSection`	371	Mapped views
`NtUnmapViewOfSectionEx`	348	Unmapped views

One memory-write target was PID 8952 (the cmd.exe child) at a logged address and size of only four bytes. Treating this as a second-stage payload injection without inspecting memory contents, executable page transitions and thread start would overstate the trace. CAPE signature names such as injection_write_process should be reported with that caveat.

Observed worker start addresses

TIME	THREAD ID	START ADDRESS	ORIGIN
01:25:10,535	4896	0x07637c10	sample / unknown
01:25:10,551	3756	0x07638e7f	sample / unknown
01:25:10,551	5936	0x0763a0fe	sample / unknown
01:25:10,582	1780	0x0763a274	sample / unknown
01:25:10,582	4192	0x07631e51	sample / unknown
01:25:10,598	8924	0x07638e04	sample / unknown
01:25:10,613	1172	0x73a67930	mswsock.dll
01:25:10,738	4112	0x7455df70	p9np.dll
01:25:10,785	1348	0x0763a073	sample / unknown

Several starting addresses line up with the low-address portion of the image rebased in the runtime process, and others belong to system DLLs. A definitive worker-to-Ghidra function mapping should normalize the DLL load base and assess the export/rebase relationship. The companion reconstructed .c preserves relevant original Ghidra addresses for this pivot.

17. The boot-stage question: hypotheses and discriminators

The absence of the expected boot-stage screen is a well-posed forensic question. The available evidence has two parts: (a) the source contains raw-volume/physical-drive procedures, and (b) the runtime trace contains no successful raw-disk write attributable to the primary process. A normal subsequent Windows boot is additional user-observed context, but it cannot tell us which branch failed.

Hypothesis matrix

HYPOTHESIS	WHY PLAUSIBLE	WHAT WOULD DISCRIMINATE IT
Privilege-gate not satisfied	0x10007deb tests a named privilege bit; some token opens denied.	Captured DAT_1001f144, or a branch trace.
Raw device could not be opened or written	Disk helper contains CreateFile/WriteFile-style path.	Disk I/O or pre/post boot-sector comparison.
Boot layout differs from stage assumptions	A successful Windows reboot is compatible with an unmodified or bypassed legacy boot path.	Disk partition/firmware metadata in archived VM configuration.
Payload branch incomplete	Worker paths involve conditions, scheduling and asynchronous execution.	Aligned thread trace and persisted post-run state.
Monitor lacks full visibility	User-mode API capture has blind spots.	Independent kernel/block telemetry or offline disk image comparison.

Why an unconditional wiper claim would be wrong

The static helper `FUN_10008d5a` references the logical system volume, and its subordinate `FUN_10008cbf` references a physical drive. These functions include raw-write APIs, but **existence does not demonstrate execution**. None of the five attributable NtWriteFile events reached those devices. We cannot decide from this record whether a failed privilege test prevented entering the branch, a separate device operation failed silently, or the monitor omitted a kernel path.

Severity and coverage

CAPE assigned malicious status and a 10/10 score. That automated classification is well supported by multiple malicious behaviors, but its severity is a classifier verdict rather than a proof that every theoretically destructive feature ran. It is defensible to report that the sample reached the main dispatcher, wiped a local staged copy, created an auxiliary PE-like file, used CryptoAPI, attempted network communication, and tried scheduling a reboot. It is **not** defensible to call MBR infection successful in this execution.

18. Background-process and false-attribution audit

The task contains a busy Windows VM. Some monitored records are Microsoft Edge update tasks, OneDrive update tasks, taskhostw, service hosts, an update subsystem and analyzer-hosted Python files. CAPE reports 33 process entries. Parent-child association is a necessary first filter, but process ancestry can be broken by scheduled tasks and service brokers; therefore even parentage is not sufficient for an unqualified malware attribution.

PID	PARENT	IMAGE	API CALLS	ATTRIBUTION
4900	6696	rundll32.exe	11889	Direct sample
8952	4900	cmd.exe	192	Direct child
6572	8952	schtasks.exe	220	Task creation child
1900	684	svchost.exe	311	Unproven relation / VM background

PID	PARENT	IMAGE	API CALLS	ATTRIBUTION
6672	1900	taskhostw.exe	1775	Unproven relation / VM background
6616	1900	dxgiadaptercache.exe	988	Unproven relation / VM background
1992	1900	taskhostw.exe	2261	Unproven relation / VM background
1640	1900	updater.exe	1415	Unproven relation / VM background
6496	1640	updater.exe	418	Unproven relation / VM background
4960	1900	wermgr.exe	242	Unproven relation / VM background
7628	1900	taskhostw.exe	2609	Unproven relation / VM background
5352	1900	taskhostw.exe	424	Unproven relation / VM background
4020	1900	updater.exe	1422	Unproven relation / VM background
1104	4020	updater.exe	415	Unproven relation / VM background
6384	1900	MicrosoftEdgeUpdate.exe	1825	Unproven relation / VM background
1068	1900	UIEOrchestratorStub.exe	26	Unproven relation / VM background
4400	1900	usoclient.exe	415	Unproven relation / VM background
1120	1900	updater.exe	1417	Unproven relation / VM background
4000	1900	taskhostw.exe	3433	Unproven relation / VM background
2300	1900	OneDriveStandaloneUpdater.exe	765	Unproven relation / VM background
3440	1900	Microsoft.Data.UsageAndQuality ...	154	Unproven relation / VM background
5400	1900	taskhostw.exe	12817	Unproven relation / VM background
8640	1900	sc.exe	147	Unproven relation / VM background
4716	1900	OneDriveStandaloneUpdater.exe	770	Unproven relation / VM background
7100	1900	devicecensus.exe	247	Unproven relation / VM background
7064	1900	wsqmcons.exe	124	Unproven relation / VM background
4492	1900	MicrosoftEdgeUpdate.exe	13141	Unproven relation / VM background
1092	684	svchost.exe	58	Unproven relation / VM background
6032	1900	usoclient.exe	409	Unproven relation / VM background

PID	PARENT	IMAGE	API CALLS	ATTRIBUTION
3124	1900	rundll32.exe	246	Unproven relation / VM background
6836	1120	updater.exe	426	Unproven relation / VM background
4260	1092	dllhost.exe	742	Unproven relation / VM background
6344	1092	SoftLandingTask.exe	14500	Unproven relation / VM background

Taxonomy of 102 dropped-file records

The CAPE dropped array contains **102 entries**, but a sizable share are recognizable guest analyzer components (for example modules under a transient analyzer folder and Python bytecode). This means a raw dropped-file count is not an accurate summary of NotPetya payload production. The most defensible immediate primary artifact from monitored PID 4900 file-write calls is `C:\ProgramData\dllhost.dat`; it should be separately recovered and hashed.

19. Automated findings: handling CAPE signature labels

All 56 signature names and severities below come from the CAPE structured report. They are presented to preserve auditability, **not** to endorse every signature label as an independently proven capability. In particular, “mass_data_encryption”, “persistence_autorun_tasks”, “hardware_id_profiling” and other broad categories may be triggered by a narrower set of indicators than a human analyst would require.

DETECTION / SIGNATURE	SEVERITY	EVIDENCE HANDLING
antivm_checks_available_memory	1	Recheck corresponding API or file trace; automated finding only
queries_computer_name	1	Recheck corresponding API or file trace; automated finding only
dynamic_function_loading	1	Recheck corresponding API or file trace; automated finding only
queries_keyboard_layout	1	Recheck corresponding API or file trace; automated finding only
queries_locale_api	1	Recheck corresponding API or file trace; automated finding only
overlay	1	Recheck corresponding API or file trace; automated finding only
static_authenticode	1	Recheck corresponding API or file trace; automated finding only
accesses_public_folder	1	Recheck corresponding API or file trace; automated finding only
antidebug_setunhandledexceptionfilter	1	Recheck corresponding API or file trace; automated finding only

DETECTION / SIGNATURE	SEVERITY	EVIDENCE HANDLING
generates_crypto_key	1	Recheck corresponding API or file trace; automated finding only
stealth_timeout	1	Recheck corresponding API or file trace; automated finding only
language_check_registry	1	Recheck corresponding API or file trace; automated finding only
privilege_elevation_check	2	Recheck corresponding API or file trace; automated finding only
dead_connect	2	Recheck corresponding API or file trace; automated finding only
dllload_suspicious_directory	2	Recheck corresponding API or file trace; automated finding only
registers_vectored_exception_handler	2	Recheck corresponding API or file trace; automated finding only
per_file_acl_token_check	2	Recheck corresponding API or file trace; automated finding only
http_request	2	Recheck corresponding API or file trace; automated finding only
creates_suspended_process	2	Recheck corresponding API or file trace; automated finding only
resumethread_remote_process	2	Recheck corresponding API or file trace; automated finding only
injection_write_process	2	Recheck corresponding API or file trace; automated finding only
reads_memory_remote_process	2	Recheck corresponding API or file trace; automated finding only
enumerates_running_processes	2	Recheck corresponding API or file trace; automated finding only
process_interest	2	Recheck corresponding API or file trace; automated finding only
reads_self	2	Recheck corresponding API or file trace; automated finding only
terminates_remote_process	2	Recheck corresponding API or file trace; automated finding only
packer_entropy	2	Recheck corresponding API or file trace; automated finding only

DETECTION / SIGNATURE	SEVERITY	EVIDENCE HANDLING
invalid_authenticode_signature	2	Recheck corresponding API or file trace; automated finding only
contains_pe_overlay	2	Recheck corresponding API or file trace; automated finding only
uses_windows_utilities	2	Recheck corresponding API or file trace; automated finding only
uses_windows_utilities_to_create_scheduled_task	2	Recheck corresponding API or file trace; automated finding only
antisandbox_sleep	3	Recheck corresponding API or file trace; automated finding only
antisandbox_unhook	3	Recheck corresponding API or file trace; automated finding only
hardware_id_profiling	3	Recheck corresponding API or file trace; automated finding only
antivm_display	3	Recheck corresponding API or file trace; automated finding only
mountpoint_manager_access	3	Recheck corresponding API or file trace; automated finding only
interprocess_comms_mutex	3	Recheck corresponding API or file trace; automated finding only
interprocess_comms_shared_memory	3	Recheck corresponding API or file trace; automated finding only
unbacked_etw_patching	3	Recheck corresponding API or file trace; automated finding only
unbacked_process_mitigation_alteration	3	Recheck corresponding API or file trace; automated finding only
unbacked_api_resolution	3	Recheck corresponding API or file trace; automated finding only
unbacked_library_load	3	Recheck corresponding API or file trace; automated finding only
unbacked_memory_protection_alteration	3	Recheck corresponding API or file trace; automated finding only
unbacked_process_creation	3	Recheck corresponding API or file trace; automated finding only
unbacked_crypto_operations	3	Recheck corresponding API or file trace; automated finding only

DETECTION / SIGNATURE	SEVERITY	EVIDENCE HANDLING
unbacked_delay_execution	3	Recheck corresponding API or file trace; automated finding only
unbacked_file_dropping	3	Recheck corresponding API or file trace; automated finding only
unbacked_service_manipulation	3	Recheck corresponding API or file trace; automated finding only
unbacked_token_manipulation	3	Recheck corresponding API or file trace; automated finding only
unbacked_memory_network_connection	3	Recheck corresponding API or file trace; automated finding only
persistence_autorun_tasks	3	Recheck corresponding API or file trace; automated finding only
mass_data_encryption	3	Recheck corresponding API or file trace; automated finding only
binary_yara	3	Recheck corresponding API or file trace; automated finding only
procmem_yara	3	Recheck corresponding API or file trace; automated finding only
folder_enumeration	3	Recheck corresponding API or file trace; automated finding only
suspicious_command_tools	3	Recheck corresponding API or file trace; automated finding only

Critical signal versus false certainty

A severity 3 match can be useful as an investigator queue, but it is not an independent network packet, a reliable instruction trace, or a verified altered sector. For example, a scheduled task command line is not the same as confirmed persistence; a CryptoAPI encryption call is not a count of encrypted data objects; and a protected memory page is not necessarily a malicious payload.

20. Ghidra call-graph triage with exact symbol anchors

To make the companion .c useful for follow-up reversing, the following table records selected original Ghidra symbol labels, the code-level interpretation, and directly invoked helper symbols. The helper calls are extracted from the submitted decompilation; only the readable role text is an analyst label.

GHIDRA SYMBOL	READABLE ANALYST ROLE	EXAMPLE DIRECT CALLEES
FUN_10007cc0	Initialize environment and privilege mask	FUN_100081ba, FUN_10008677, FUN_10008acf
FUN_100081ba	Test and request named privilege	-

GHIDRA SYMBOL	READABLE ANALYST ROLE	EXAMPLE DIRECT CALLEES
FUN_10007deb	Exported primary workflow	FUN_10007cc0, FUN_10009590, FUN_10007091, FUN_10006a2b
FUN_10009590	Allocate/relocate loaded image	FUN_10009322, FUN_100091fa, FUN_10009286
FUN_100094a5	Original DLL overwrite/cleanup path	FUN_10009367, FUN_10007deb
FUN_10009367	Rebind imported procedures	-
FUN_10009286	Adjust memory protections	-
FUN_10008677	Process enumeration / context	-
FUN_10008320	System marker path logic	-
FUN_1000835e	System-marker check	FUN_10008320
FUN_10008d5a	Raw-volume stage (static only)	FUN_100014a9, FUN_10008cbf
FUN_10008cbf	Raw physical-drive substage (static only)	-
FUN_100084df	Scheduled reboot orchestration	FUN_10006973, FUN_10008494, FUN_100083bd
FUN_100083bd	Launch command helper	-
FUN_10007545	Embedded auxiliary stage	FUN_100085d0, FUN_100073ae, FUN_100070fa
FUN_10008999	Resource extraction / helper drop	FUN_100085d0, FUN_10008946
FUN_10007c10	Background discovery worker	FUN_10006fc7, FUN_1000777b, FUN_1000786b, FUN_1000795a
FUN_10001eef	Logical drive enumeration	-
FUN_10001e51	Per-volume cryptographic worker	FUN_10001b4e, FUN_10001973, FUN_10001d32
FUN_10008282	Timing and host-dependent branches	FUN_10006973
FUN_1000a0fe	Coordination worker	FUN_10009f8e, FUN_10006f40, FUN_10006f78, FUN_10006f02
FUN_1000a274	Delayed worker	FUN_10006f40, FUN_10009dc3, FUN_10006f91, FUN_10006f02

Why this is an analytical reconstruction, not recovered source

The Ghidra output has synthetic symbols and nonstandard artifacts: overlapping globals, ambiguous structural types, inferred size_t and HANDLE values, and some apparent mismatches between argument types and imported prototypes. The companion reconstruction avoids claiming an exact original source layout, original variable names, original control-flow fidelity for unverified functions, or repeatable malicious behavior.

21. Reproducibility and next-evidence protocol

Read-only report queries

The following queries are safe on the already exported CAPE JSON. They do not run, modify, load or reexecute the malicious DLL. They let an independent reviewer verify the strongest findings without trusting the narrative.

```
import json, collections
R = json.load(open('report.json', encoding='utf-8'))
P = next(p for p in R['behavior']['processes'] if p['process_id']==4900)
N = collections.Counter(c['api'] for c in P['calls'])
print('Primary events:',len(P['calls']))
for key in ['NtOpenProcessToken','NtWriteFile','CryptEncrypt','connect']:
    print(key,N.get(key,0))
for c in P['calls']:
    if c['api']=='NtWriteFile':
        print(c['timestamp'],[(a['name'],str(a['value'])[:95])
               for a in c['arguments'] if a['name'] in ('HandleName','Length')])
```

Evidence collection that would resolve the open issues

QUESTION	BEST ARTIFACT	WHY THE PRESENT RECORD IS INSUFFICIENT
Did the first sector change?	Baseline vs post-run virtual-disk images	User-mode call traces cannot certify sector state.
What was the privilege mask?	Saved process memory or branch observation	Token open success/denial is not the same as privilege enablement.
What was dllhost.dat?	Validated extracted dropped PE	MZ header establishes format candidate, not capabilities.
Which files changed?	Pre/post file hashes + matching write handles	API encryption calls do not enumerate finished encrypted outputs.
Was propagation successful?	Remote host evidence / full PCAP / account logs	191 unsuccessful connects and one failed admin$ operation.
Was task installed?	Task Scheduler event XML or task store	Command invocation alone does not prove persisted task.

Damaged archive: impact and correction

The submitted `cape-task4.tar.gz` contains a prematurely terminated gzip stream. The report.json and two logs were recovered, but memory, EVTX, screenshots, some dropped-file bytes and PCAP raw contents were not independently validated. Therefore every statement based on network/session summary, CAPE extraction metadata or memory detections is explicitly a **reported finding**. The PDF does not claim that the unavailable artifacts were separately reverse engineered.

Final evidentiary synthesis

The sample was invoked through ordinal #1; the main worker demonstrably ran, touched its own image, wrote a separate MZ-format auxiliary file, created and resumed a command child, attempted to schedule a restart, called CryptoAPI and attempted local/network-service communication. The malware-associated API trace does **not** show physical-drive writing; the reboot-stage outcome remains unresolved. Denied token requests are important but not a sufficient root-cause explanation. These distinctions represent the strongest conclusion available without a complete forensic image.

22. Decompiled flow-to-telemetry map

GHIDRA ADDRESS	READABLE ROLE	EVIDENCE
entry / 0x10007d39	DLL attach initialization	Static
FUN_10007deb	Ordinal-1 primary dispatcher	Static + process execution
FUN_10007cc0	Privilege flag initialization	Static
FUN_100081ba	Attempt named privilege enable	Static + token events
FUN_10009590	Image copy/relocation path	Static
FUN_100094a5	Self-image wipe and cleanup	Static + file write
FUN_1000835e	System path/marker check	Static
FUN_10008d5a	Raw C: volume operation branch	Static; no observed raw write
FUN_10008cbf	PhysicalDrive0 operation substage	Static; no observed raw write
FUN_100084df	Scheduled reboot helper	Static + child chain
FUN_100083bd	Command-process launching helper	Static + cmd.exe
FUN_10008999	Resource extraction / aux file	Static + dllhost.dat write
FUN_10007c10	Network discovery worker	Static + network attempts
FUN_10001eef	Fixed-drive / crypto workers	Static + crypto activity
FUN_10008282	Timing and host-dependent delays	Static
FUN_1000a0fe	Worker/coordination thread	Static

For the expanded function-level reconstruction, open the accompanying C text. Each conceptual function has a header mapping back to its original FUN_... symbol, source/dynamic confidence annotations, and a narrative description of branch

purpose.

Methodological warning for decompiled code

Ghidra output may contain incorrect inferred types, calls with misidentified arguments, overlapping globals, and “uninitialized” locals that are artifacts of analysis. The flow map is an interpretive model and must not be described as an exact recovered source tree.

23. Open questions and forensic next steps

Boot-sector difference

Compare the baseline disk boot sectors and the post-run virtual-disk image offline. A checksum-level comparison is stronger than inferring from an eventual successful reboot.

Privilege result

Use an existing debugger trace or preserved memory to determine DAT_1001f144 after FUN_10007cc0 and distinguish successful token opens from successfully enabled privileges.

Device I/O visibility

Cross-check guest ETW/Sysmon/ProcMon and disk images for raw-volume or PhysicalDrive0 writes; the CAPE API monitor alone may not cover every kernel I/O path.

Auxiliary PE

Recover the 381,816-byte dllhost.dat artifact from an intact task archive; inspect its PE headers, exports, sections and hash without executing it.

Cryptography scope

Correlate CryptEncrypt calls with file names, buffers and thread context before claiming any number of encrypted files.

Process attribution

Remove analyzer scripts, Windows scheduled maintenance and automatic update processes from malware-specific metrics unless direct lineage or call-site evidence exists.

Archive recovery

Acquire a fresh, validated archive before deeper memory, EVTX or packet interpretation. Current .tar.gz has an unexpected EOF.

What this report deliberately does not claim

It does not claim a confirmed successful MBR replacement, persistent scheduled task, completed ransomware encryption, credential theft, or remote infection. It does not provide an operational reenactment of destructive write, exploit or worm propagation. These are distinct evidentiary questions.

Appendix A. Evidence locations and reproducibility

CLAIM	RECORD / PATH
File hash, format, section count	notpetya(1).txt: identification, SHA-256, binwalk sections
Task setup/duration/package	report.json: info, target.file
Primary process and child chain	report.json: behavior.processes, behavior.processtree
Individual token and write events	report.json: behavior.processes[PID=4900].calls
Task Scheduler command	report.json: behavior.summary.executed_commands
CryptEncrypt and connect count	report.json: PID 4900 calls, grouped by api/status
Packet/network view	report.json: network; no extracted PCAP bytes reviewed
Raw disk branch and privilege gate	NotPetya.dll.c: FUN_10007cc0, FUN_100081ba, FUN_10007deb, FUN_10008d5a, FUN_10008cbf
Self-wipe and resources	NotPetya.dll.c: FUN_100094a5, FUN_10008999

Selected raw event statuses

```
PID 4900 NtOpenProcessToken   desired=0x00000080 -> 0xC0000022
PID 4900 NtOpenProcessToken   desired=0x00000028 -> 0xC0000022
PID 4900 NtOpenProcessToken   desired=0x00000008 -> SUCCESS
PID 4900 NtOpenProcessToken   desired=0x0000000A -> SUCCESS
PID 4900 NtWriteFile          Temp\NotPetya.dll -> SUCCESS
PID 4900 NtWriteFile          C:\ProgramData\dllhost.dat -> SUCCESS
PID 4900 NtCreateFile         remote admin$ path -> 0xC00000BE
PID 4900 CryptEncrypt         346 calls -> 346 successes
PID 4900 connect              191 calls -> 0 successes
```

Checks are based on CAPE’s reported success/status values. Return-code descriptions have been normalized for readability. The output intentionally omits sample bytes, recovered credential material and any functional wiper routines.

References within supplied materials

1. CAPE Task 4, recovered report.json (analysis date: 6 November 2026).
2. Ghidra C-like export, NotPetya.dll.c (12,031 lines).
3. Static triage dump, notpetya(1).txt (file/digest/import/string/entropy evidence).
4. Reconstructed companion file, NotPetya_Reconstructed_Annotated_Flow_EN.c (research mapping, not original source).